Name : Aman Raychaudhuri
Sem : 5
Div : T2 Batch - T21
Roll No:2401094
Sub:ASD&D

Experiment 2: To Containerize Application using Docker

Aim:

To containerize application using Docker

This maps to LO4.

LO4: Containerize, orchestrate and deploy applications using Docker, Docker Compose, Kubernetes and cloud infrastructure tools.

Theory:

Steps to install docker:

i. Navigate to the official Docker website (docker.com) and download the Docker Desktop installer for your operating system (Windows).
ii. Locate the downloaded installer file on your computer and double-click it to run the setup wizard.
iii. Follow the on-screen prompts. When asked, ensure the option to Use WSL 2 instead of Hyper-V is checked (for optimal performance on Windows).
iv. Wait for the installation to finish, click "Close," and restart your computer if the installer prompts you to do so.
v. After restarting, search for "Docker Desktop" in your system's Start menu and launch the application. Accept the service agreement to start the Docker engine.
vi. Open your terminal or PowerShell and type docker --version and press Enter. If it returns a version number, Docker is successfully installed and ready to use.

Docker:

Docker is an open-source platform that allows you to package an application and all its required dependencies into a single, standardized unit for software development. This ensures that the application runs exactly the same way regardless of where it is deployed—whether on your personal laptop, a testing environment, or a live production server. Instead of dealing with the old problem of "it works on my machine but not yours," Docker isolates the software so it behaves predictably everywhere.

Docker Images: A Docker image is a lightweight, standalone, and immutable (read-only) executable package. In theory, it is a static file that contains the complete set of instructions and components necessary to run a specific application.

An image is built from multiple stacked layers, where each layer represents a specific instruction—such as installing a dependency, setting an environment variable, or copying application code. It encompasses the application code itself, the runtime environment, system tools, system libraries, and default configuration settings. Because it is read-only, an image cannot be changed once it is built.

Docker Containers: A container is the active, runtime execution of a Docker image. It is a standardized, isolated operating environment where the application actually runs and processes data.

In theory, when an image is executed, the Docker engine adds a thin, writable layer on top of the static image layers. This writable layer allows the active container to modify files, generate logs, and maintain a temporary state while it is running. Containers isolate the running software from the host operating system and from other containers, which ensures the application performs uniformly and predictably regardless of the underlying infrastructure.

Implementation:

1. docker --version

```
C:\Users\203>docker --version
Docker version 25.0.3, build 4debf41
```

2. docker images

```
C:\Users\203>docker images
REPOSITORY        TAG         IMAGE ID       CREATED          SIZE
aryank            latest      cd0827da2eff   26 hours ago     132MB
<none>            <none>      42f468bdae9d   26 hours ago     132MB
nginx             latest      4e5db4761e0f   6 days ago       161MB
python            3.12-slim   25c5b8011a34   8 days ago       119MB
ubuntu            latest      de7345b16e94   8 days ago       100MB
my-nginx          latest      c18468aa2adf   3 months ago     152MB
hello-world       latest      e2ac70e7319a   4 months ago     10.1kB
nginx             <none>      9d0e6f6199dc   8 months ago     152MB
jenkins/jenkins   lts         6a44d1dd2d60   16 months ago    468MB
docker            dind        a16883c00816   17 months ago    395MB
node              latest      8bbb9d1bc633   2 years ago      1.1GB
sample-web-app    latest      d03cd9229681   2 years ago      912MB
gradle            latest      716c875ae781   2 years ago      713MB
postgres          latest      eb634efa7ee4   2 years ago      431MB
redis             <none>      d1397258b209   2 years ago      138MB
redis             latest      170a1e90f843   2 years ago      138MB
```

docker

3. docker search postgress

```
C:\Users\203>docker search postgres
NAME                      DESCRIPTION                                     STARS     OFFICIAL
postgres                  The PostgreSQL object-relational database sy…   14969     [OK]
cimg/postgres                                                             9
circleci/postgres         The PostgreSQL object-relational database sy…   35
kasmweb/postgres          Postgres image maintained by Kasm Technologi…   6
elestio/postgres          Postgres, verified and packaged by Elestio      2
ubuntu/postgres           PostgreSQL is an open source object-relation…   45
cleanstart/postgres       Secure by Design, Built for Speed, Hardened …   0
artifacthub/postgres                                                      0
geokrety/postgres         Postgres with postgis + quantile and amqp ex…   0
corpusops/postgres        https://github.com/corpusops/docker-images/     0
rootpublic/postgres                                                       0
dockette/postgres         My PostgreSQL image with tunning and preinst…   1
vulhub/postgres                                                           2
wayofdev/postgres                                                         0
pgrouting/postgres         Postgres Docker images with PostGIS and dep…   0
uselagoon/postgres                                                        0
openeuler/postgres                                                        0
clarinpl/postgres                                                         0
mosipdev/postgres                                                         0
supabase/postgres         Unmodified Postgres with some useful plugins…   78
brimstone/postgres        postgres image with traefik-cert support        0
trainlineeurope/postgres  Extended version of official Postgres https:…   0
blacklabelops/postgres    Postgres Image for Atlassian Applications       4
fredboat/postgres         PostgreSQL 10.0 used in FredBoat's docker-co…   1
sushantdhiman/postgres    Postgres Image for Sequelize                    2
```

4. docker search --filter stars=100 ubuntu

```
C:\Users\203>docker search --filter stars=100 ubuntu
NAME              DESCRIPTION                                     STARS     OFFICIAL
ubuntu            Ubuntu is a Debian-based Linux operating sys…   17859     [OK]
ubuntu/squid      Squid is a caching proxy for the Web. Long-t…   129
ubuntu/nginx      Nginx, a high-performance reverse proxy & we…   141
ubuntu/apache2    Apache, a secure & extensible open-source HT…   104
ubuntu/bind9      BIND 9 is a very flexible, full-featured DNS…   120
```

5. docker run -it ubuntu /bin/bash

```
C:\Users\203>docker run -it ubuntu /bin/bash
root@64fd50f6a69e:/# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@64fd50f6a69e:/# exit
exit
```

6. mkdir docker-demo & cd docker-demo

```
C:\Users\203>mkdir docker-demo

C:\Users\203>cd docker-demo
```

7. docker build -t flask-app .

```
C:\Users\203\docker-demo>docker build -t flask-app .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  4.096kB
Step 1/7 : FROM python:3.12-slim
 ---> 25c5b8011a34
Step 2/7 : WORKDIR /app
 ---> Using cache
 ---> ed8efe307646
Step 3/7 : COPY requirements.txt .
 ---> Using cache
 ---> ee6056b81239
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
 ---> Using cache
 ---> afd4198ae7cb
Step 5/7 : COPY . .
 ---> cfbc6486ee17
Step 6/7 : EXPOSE 5000
 ---> Running in cc339862ff5e
 ---> Removed intermediate container cc339862ff5e
 ---> 9675e30ec2a9
Step 7/7 : CMD ["python", "app.py"]
 ---> Running in f366060e9178
 ---> Removed intermediate container f366060e9178
 ---> e999c1ddded2
Successfully built e999c1ddded2
Successfully tagged flask-app:latest
SECURITY WARNING: You are building a Docker image from Windows against a non-Windows Docker host. All
files and directories added to build context will have '-rwxr-xr-x' permissions. It is recommended to
double check and reset permissions for sensitive files and directories.

What's Next?
  1. Sign in to your Docker account → docker login
  2. View a summary of image vulnerabilities and recommendations → docker scout quickview
```

8. docker images

docker

```
C:\Users\203\docker-demo>docker images
REPOSITORY        TAG         IMAGE ID       CREATED         SIZE
flask-app         latest      e999c1ddded2   3 minutes ago   132MB
aryank            latest      cd0827da2eff   27 hours ago    132MB
<none>            <none>      42f468bdae9d   27 hours ago    132MB
nginx             latest      4e5db4761e0f   6 days ago      161MB
python            3.12-slim   25c5b8011a34   8 days ago      119MB
ubuntu            latest      de7345b16e94   8 days ago      100MB
my-nginx          latest      c18468aa2adf   3 months ago    152MB
hello-world       latest      e2ac70e7319a   4 months ago    10.1kB
nginx             <none>      9d0e6f6199dc   8 months ago    152MB
jenkins/jenkins   lts         6a44d1dd2d60   16 months ago   468MB
docker            dind        a16883c00816   17 months ago   395MB
node              latest      8bbb9d1bc633   2 years ago     1.1GB
sample-web-app    latest      d03cd9229681   2 years ago     912MB
gradle            latest      716c875ae781   2 years ago     713MB
postgres          latest      eb634efa7ee4   2 years ago     431MB
redis             latest      170a1e90f843   2 years ago     138MB
redis             <none>      d1397258b209   2 years ago     138MB
```

9. docker run -d -p 5000:5000 flask-app

```
C:\Users\203\docker-demo>docker run -d -p 5000:5000 flask-app
4dfb4198b723e1e7162cae65c427c94c43057eca56781431a36500060beebe1c
```

10. docker ps

```
C:\Users\203\docker-demo>docker ps
CONTAINER ID   IMAGE       COMMAND           CREATED              STATUS              PORTS
        NAMES
4dfb4198b723   flask-app   "python app.py"   About a minute ago   Up About a minute   0.0.0.0:5000->50
00/tcp   stoic_mcclintock
```

11. docker logs 4dfb4198b723

```
C:\Users\203\docker-demo>docker logs 4dfb4198b723
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
 server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [22/Jul/2026 08:24:13] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [22/Jul/2026 08:24:13] "GET /favicon.ico HTTP/1.1" 404 -
```

12. app serving

Hello Docker!

13. docker stop 4dfb4198b723

```
C:\Users\203\docker-demo>docker stop 4dfb4198b723
4dfb4198b723
```

docker

14. docker ps

```
C:\Users\203\docker-demo>docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

15. docker rm 4dfb4198b723

```
C:\Users\203\docker-demo>docker rm 4dfb4198b723
4dfb4198b723
```

16. docker ps -a

```
C:\Users\203\docker-demo>docker ps -a
CONTAINER ID   IMAGE                  COMMAND                  CREATED          STATUS
       PORTS                                  NAMES
64fd50f6a69e   ubuntu                 "/bin/bash"              39 minutes ago   Exited (0) 33 minutes a
go                                            nice_lalande
3aa03439b164   aryank                 "python app.py"          27 hours ago     Exited (255) 42 minutes
 ago   0.0.0.0:5001->5000/tcp                 aryank-background
9b433640960c   ubuntu                 "/bin/bash"              28 hours ago     Exited (0) 28 hours ago
                                              eager_bartik
79d59be7b7ef   ubuntu                 "/bin/bash"              28 hours ago     Exited (0) 28 hours ago
                                              quirky_matsumoto
c8fe146dd791   ubuntu                 "bash"                   46 hours ago     Exited (255) 28 hours a
go                                            pedantic_hypatia
2ba1c97abb52   ubuntu                 "/bin/bash"              47 hours ago     Exited (127) 47 hours a
go                                            suspicious_hugle
0fb51210958c   my-nginx               "/docker-entrypoint.…"   3 months ago     Exited (255) 47 hours a
go     0.0.0.0:9091->80/tcp                   thirsty_pare
fe399986e504   9d0e6f6199dc           "/docker-entrypoint.…"   3 months ago     Exited (255) 47 hours a
go     0.0.0.0:9090->80/tcp                   infallible_germain
5be8ea3f3bb2   9d0e6f6199dc           "/docker-entrypoint.…"   3 months ago     Created
                                              ecstatic_jennings
1e61b2cb20bf   hello-world            "/hello"                 3 months ago     Exited (0) 3 months ago
                                              nervous_elbakyan
23106efee7f4   9d0e6f6199dc           "/docker-entrypoint.…"   8 months ago     Exited (255) 4 months a
go     0.0.0.0:8081->80/tcp                   my-nginx
d1c21cfdc92b   jenkins/jenkins:lts    "/usr/bin/tini -- /u…"   16 months ago    Exited (255) 8 months a
go     50000/tcp, 0.0.0.0:9090->8080/tcp      jenkins
6d895ae2b5eb   d1397258b209           "docker-entrypoint.s…"   2 years ago      Exited (0) 2 years ago
                                              nice_greider
1f99a9cc77bf   gradle                 "/__cacert_entrypoin…"   2 years ago      Exited (0) 2 years ago
                                              elastic_stonebraker
77e3fc5c920f   node                   "docker-entrypoint.s…"   2 years ago      Exited (0) 2 years ago
                                              con234
7b58e9788533   node                   "docker-entrypoint.s…"   2 years ago      Exited (0) 2 years ago
                                              containerblah
0dc3412adadd   postgres               "docker-entrypoint.s…"   2 years ago      Exited (1) 2 years ago
                                              stoic_mclaren
```

docker

17. docker rmi flask-app

```
C:\Users\203\docker-demo>docker rmi flask-app
Untagged: flask-app:latest
Deleted: sha256:e999c1ddded2b14c0410d4bba84f979a61819dd920c6db4759546992e8919b48
Deleted: sha256:9675e30ec2a96bebfdad54d5f5c507189d42aeb58c76071c5139553c05db6d97
Deleted: sha256:cfbc6486ee1795fd46a8f7ca3a1daa06c036c338398c7acc1822f7d6dbb1b53f
Deleted: sha256:6012b1d999f5c448de14101f15f09162140ec8c3b1aeff51bf1172a8ee934099
```

Conclusion:

The experiment on **Containerizing an Application Using Docker** was successfully completed. Docker was used to build a Docker image for a Flask application using a Dockerfile, which packaged the application along with its required dependencies into a portable image. The image was successfully created and executed as a Docker container with the appropriate port mapping, allowing the application to be accessed through a web browser. The container's execution was verified using Docker commands such as docker ps and docker logs, confirming that the application was running correctly and responding to HTTP requests. Finally, the container was stopped, removed, and the Docker image was deleted, demonstrating the complete Docker workflow. This experiment highlighted how Docker simplifies application deployment by providing a consistent, isolated, and portable execution environment across different systems.